

AWS Golden Jacket Project Report

Serverless Observability & Distributed Tracing (DVA-C02 Focus)

AWS Golden Jacket Coach

June 27, 2026

1. Executive Summary & Business Problem

Business Problem: A school district required a highly scalable, serverless portal separating Student, Teacher, and Admin access. The operational team needed deep visibility into system latency, database bottlenecks, and application errors to maintain strict SLAs.

Architecture Overview: A Serverless Microservices pattern using Amazon API Gateway (REST), AWS Lambda (Compute), and Amazon DynamoDB (Data). Observability was implemented using AWS X-Ray and Amazon CloudWatch.

2. Architecture & AWS Services

AWS Services Used & Rationale:

- **Amazon API Gateway:** Acts as the front door. Chose `AWS_PROXY` integration to pass the entire HTTP request to Lambda, standardizing request handling.
- **AWS Lambda:** Serverless compute. Separated into 3 functions (Student, Teacher, Admin) to isolate failure domains and enforce least privilege.
- **Amazon DynamoDB:** Chose On-Demand billing for unpredictable traffic. Implemented Single-Table Design using generic PK/SK to prevent hot-partitions and optimize cost.
- **AWS X-Ray:** Chosen over CloudWatch Logs alone because it stitches together the entire distributed request (Trace) to identify latency bottlenecks.

3. Infrastructure as Code (IaC) & Deployment

Summary: Deployed via AWS CloudFormation across three isolated layers (Data, Compute, API) using Cross-Stack References (`Export / Fn::ImportValue`).

- **Gotcha:** CloudFormation inline `ZipFile` does not support external `node_modules`. Including `aws-xray-sdk-core` without a Lambda Layer caused a fatal 500 error.
- **Solution for Production:** Use AWS SAM or package the function with a Lambda Layer containing the X-Ray SDK.

4. Production Failure Engineering & Pressure Testing

Failure Scenarios Tested:

- **Missing Module Crash (500 Error):** API Gateway returned a 500 error because the Lambda failed to initialize. Diagnosed via CloudWatch Logs execution errors.
- **Latency Injection (slowPath):** Injected a 3-second sleep. X-Ray proved the latency was in the Compute layer, not the Database layer.
- **Scan vs. Query Cost:** Demonstrated that Admin Scan consumes exponentially more read capacity units than the Teacher Query, proving the danger of Scans in production.

5. AWS Well-Architected & Security Review

- **Security (IAM):** Strict least privilege. Student Lambda role restricted to `dynamodb:Query` only. Admin Lambda allowed `dynamodb:Scan`. All required `AWSXRayDaemonWriteAccess`.
- **Reliability:** DynamoDB Point-in-Time Recovery (PITR) enabled. API Gateway proxy handles scaling inherently.
- **Operational Excellence:** TracingConfig set to `Active` on all Lambdas.

6. Certification Intelligence (DVA-C02 Exam Focus)

Domains Covered: Development with AWS Services (32%), Security (26%), Troubleshooting and Optimization (18%).

- **X-Ray Annotations vs. Metadata (CRITICAL):** Annotations are key-value pairs that are **indexed and searchable** (e.g., `studentId=S001`). Metadata is for unstructured data (e.g., a JSON payload) and is **not indexed**.
- **Lambda Proxy Integration:** In proxy integration, the Lambda function is responsible for returning a specific JSON structure: `{ "statusCode": 200, "body": "... " }`. If missing, API Gateway throws a 502 Bad Gateway.
- **DynamoDB Item Limit:** Max item size is 400KB.
- **CloudFormation Cross-Stack:** You cannot delete a stack that is exporting a value if another stack is currently importing it.

7. Observation Library & Flashcards

Observation Library:

- *API Gateway behaves as a strict proxy when using `AWS_PROXY` because it expects the backend Lambda to construct the exact HTTP response payload.*
- *DynamoDB Scans are dangerous in production because they read every item in the table, consuming massive RCUs and spiking latency as the table grows.*

Principal Architect Questions:

- "If our DynamoDB table grows to 10TB, how will the Admin dashboard endpoint perform, and how would you redesign it?" (Answer: The Scan will time out the Lambda. We must implement an Event-Driven architecture using DynamoDB Streams to aggregate totals into a separate metadata item).

8. Portfolio & LinkedIn Summary

Project Name: Serverless Observability & Distributed Tracing Engine

Summary for Resume: Architected a serverless microservices API using API Gateway, Lambda, and DynamoDB. Implemented Single-Table Design for optimal NoSQL performance. Instrumented the application with AWS X-Ray to track distributed request traces, successfully isolating code latency from database latency and resolving production deployment module faults.

9. Golden Jacket Assessment

- **Project Difficulty:** 8/10 (Complex IaC layering, advanced NoSQL concepts, observability instrumentation).
- **Associate Certification Readiness Score:** +15% bump in DVA-C02 readiness.
- **Next Steps:** To achieve true mastery, the next sprint should focus on AWS SAM (Serverless Application Model) to package this exact architecture with the X-Ray SDK inside a Lambda Layer.